MANIPAL
UNIVERSITY JAIPUR
(University under Section 2(f) of the UGC Act)

AGRICULTURAL SUPPLY CHAIN MANAGEMENT SYSTEM

RDBMS Lab Project Report

Submitted by:

VANSH PODDAR
sec – S4
RegNo: 2502052774

Under the supervision of:

DR. Ruchika Katariya

Department of Computer Science and Engineering
Manipal University Jaipur
Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway,
Jaipur, Rajasthan 303007

Abstract

The Agricultural Supply Chain Management System is a comprehensive database-driven platform developed to provide an efficient, transparent, and user-friendly platform for tracking agricultural produce from farm harvest to consumer retail shelves. The project focuses on combining an interactive web interface with a relational database to manage supply-chain-related information efficiently.

The application provides features such as farmer harvest registration, cold storage monitoring, real-time logistics fleet dispatch, wholesale and retail distribution orders, and farm-to-table consumer traceability. Its modular interface and intuitive navigation make the different stages of the agricultural supply lifecycle easily accessible.

MySQL is used as the database management system to store and organize information related to farmers, crops, harvest batches, logistics transits, and retail orders. The database consists of five normalized tables and uses primary keys, foreign keys, and enumerated domain constraints to maintain data integrity and establish relationships between records.

The project demonstrates the practical application of database design, relational schema development, SQL DDL/DML, and client-server connectivity in building a database-driven supply chain portal. It also provides hands-on experience in integrating real-world logistical workflows with structured relational data management.

Overall, the project serves as an academic implementation of an agricultural supply chain management application while demonstrating fundamental concepts of relational database management systems and modern software development.

TABLE OF CONTENTS

S. No.	Contents	Page No.
	Abstract	1
	Table of Contents	2
	List of Figures	4
	List of Tables	5
1	Introduction	6
1.1	Project Overview	6
1.2	Objectives of the Project	6
1.3	Importance and Relevance	6
1.4	Scope and Limitations	7
2	System Design & Database Architecture	8
2.1	Database Design Overview	8
2.2	Entity-Relationship (ER) Diagram	8
2.3	Entity Relationships & Cardinalities	9
2.4	Description of Tables and Attributes	10
2.5	Database Implementation: SQL DDL Code	12
2.6	Sample Data Population: SQL DML Code	13
2.7	Relational Join Queries: SQL DQL Code	14

S. No.	Contents	Page No.
3	Frontend Implementation and Screenshots	15
3.1	System Architecture Overview	15
3.2	Live Supply Chain Operations Dashboard	16
3.3	Farmer Harvest Registration Feature	17
3.4	Cold Chain Logistics and Fleet Transit	18
3.5	Retail Hub and Supermarket Orders	18
3.6	Farm-to-Table Consumer Traceability Feature	19
3.7	Java Desktop & Applet JDBC Interface Code	19
4	Conclusion & Future Scope	20
4.1	Summary of Project Work	20
4.2	Achievement of Project Objectives	20
4.3	Future Enhancements & Bibliography	21

List of Figures

Figure No.	Figure Title	Page No.
Figure 2.1	ER Diagram for Agricultural Supply Chain System	9
Figure 3.1	3-Tier System Architecture Diagram	15
Figure 3.2	Live Operations Dashboard – Inventory and KPI View	16
Figure 3.3	Farmer Harvest Registration and Producer Directory	17
Figure 3.4	Cold Chain Logistics and Dispatch Tracking	18
Figure 3.5	Retail Hub and Distribution Order Placement	18
Figure 3.6	Farm-to-Table Verified Consumer Audit Timeline	19
Figure 3.7	Java Desktop / JApplet Interface with MySQL JDBC	19

List of Tables

Table No.	Table Title	Page No.
Table 2.1	Farmers Table Schema	10
Table 2.2	Crops Table Schema	10
Table 2.3	Batches Table Schema (Core Unit)	11
Table 2.4	Logistics Table Schema	11
Table 2.5	Retail Orders Table Schema	11

Chapter 1
Introduction

1.1 Project Overview

The Agricultural Supply Chain Management System is an enterprise database application developed to provide stakeholders with a transparent and unified digital platform for tracking harvested agricultural commodities. The project focuses on implementing essential supply chain tracking functionalities along with relational database management.

The application is designed with a user-friendly interface and a structured relational database to manage information related to farmers, crops, harvest batches, transit logistics, and retail distribution. It demonstrates the practical application of database concepts and software development techniques in building a mission-critical agricultural application.

1.2 Objectives of the Project

The primary objectives of this project were as follows:

- To develop a simple and interactive supply chain management web and desktop interface.
- To design a normalized relational database for storing farmer, crop, batch, logistics, and retail records.
- To establish robust database connectivity using Java, JDBC, and RESTful APIs.
- To implement primary keys, foreign keys, and cascade rules to ensure data integrity.
- To model sequential supply chain stages: Harvested, Cold Storage, In Transit, Distributed, Retail Shelved, and Sold.
- To provide consumers with farm-to-table traceability through unique Batch IDs.
- To apply the theoretical concepts learned in Object-Oriented Programming and RDBMS.

1.3 Importance and Relevance

Agricultural supply networks are vital for national food security and economic sustainability. Post-harvest losses due to poor temperature tracking and middleman exploitation severely diminish agricultural profitability. This project provides practical experience in designing a database-driven system that ensures accountability, prevents food spoilage, and provides consumers with verified origin information. It also demonstrates how MySQL, Java, and Web technologies seamlessly integrate in production architectures.

1.4 Scope and Limitations

The current scope of the project encompasses digital farmer registration, crop batch management, cold-chain refrigerated transit recording, retail supermarket purchase orders, and consumer traceability lookups. The implementation is intended as an academic and functional prototype demonstrating robust relational database principles.

The project's limitations include simulated manual entries for IoT sensors rather than direct physical hardware telematics integration, and internal simulated billing rather than an active commercial banking payment gateway. Future iterations will consider incorporating automated IoT sensors and smart contract blockchain verification.

Chapter 2
System Design & Database Architecture

2.1 Database Design Overview

The database of the Agricultural Supply Chain Management application is designed using MySQL, an industry-standard relational database management system. It stores and organizes information related to farmers, crops, batches, logistics, and retail orders.

The database consists of five tables: farmers, crops, batches, logistics, and retail_orders. Primary keys uniquely identify each record, while foreign keys establish relationships and enforce referential integrity with ON DELETE CASCADE constraints. Enumerated types (ENUM) constrain state transitions, preventing invalid logistical statuses.

2.2 Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) diagram illustrates the core entities, attributes, and relationships within the database. The primary entities are Farmers, Crops, Batches, Logistics, and Retail Orders.

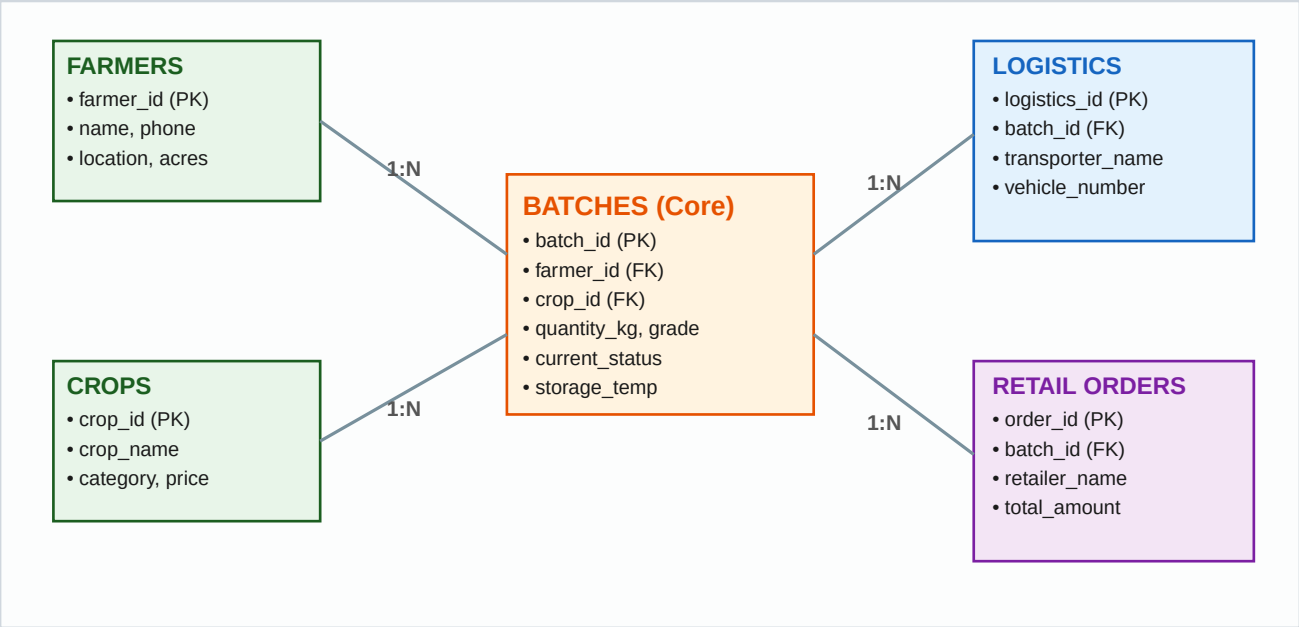

Figure 2.1: ER Diagram for the Agricultural Supply Chain System

2.3 Entity Relationships & Cardinalities

1. Farmers and Batches (1 : N)
A registered farmer can cultivate and harvest multiple batches of crops. Each batch originates from exactly one farmer. This is enforced by the farmer_id foreign key in the batches table referencing farmers(farmer_id).

2. Crops and Batches (1 : N)
A specific crop catalog item (such as Basmati Rice or Organic Wheat) categorizes multiple batches harvested across different regions. Enforced via crop_id in batches referencing crops(crop_id).

3. Batches and Logistics (1 : N)
A single batch can undergo multiple transportation legs (e.g. from farm gate to cold storage hub, and hub to wholesale terminal). Represented by the batch_id foreign key in the logistics table.

4. Batches and Retail Orders (1 : N)
A large harvest batch can be sold across multiple purchase orders to different supermarket retail chains. Enforced via batch_id in retail_orders referencing batches(batch_id).

2.4 Description of Tables and Attributes

Table 2.1: Farmers Table Schema

Field	Type	Key	Description
farmer_id	INT	PK	Unique identifier for each farmer (Auto Increment)
name	VARCHAR(100)	—	Full legal name of the producer
phone	VARCHAR(20)	—	Contact telephone number
location	VARCHAR(150)	—	Geographical farm location / District
farm_size_acres	DECIMAL(6,2)	—	Total cultivated land area in acres
created_at	TIMESTAMP	—	Record creation timestamp

Table 2.2: Crops Table Schema

Field	Type	Key	Description
crop_id	INT	PK	Unique identifier for crop type (Auto Increment)
crop_name	VARCHAR(100)	—	Name of crop (e.g. Basmati Rice, Wheat)
category	ENUM(...)	—	Grains, Vegetables, Fruits, Pulses, Spices
base_price_per_kg	DECIMAL(8,2)	—	Benchmark rate per kilogram (INR)

Table 2.3: Batches Table Schema (Core Unit)

Field	Type	Key	Description
batch_id	VARCHAR(50)	PK	Unique alphanumeric traceability code
farmer_id	INT	FK	References farmers(farmer_id)
crop_id	INT	FK	References crops(crop_id)
quantity_kg	DECIMAL(10,2)	—	Total harvested batch mass in kg
harvest_date	DATE	—	Date when crop was harvested
quality_grade	ENUM(...)	—	Grade A (Export), Grade B, Grade C
current_status	ENUM(...)	—	Harvested, Storage, Transit, Retail, Sold
storage_temp_celsius	DECIMAL(4,1)	—	Monitored storage temperature
created_at	TIMESTAMP	—	Timestamp of batch registration

Table 2.4: Logistics Table Schema

Field	Type	Key	Description
logistics_id	INT	PK	Unique shipment ID (Auto Increment)
batch_id	VARCHAR(50)	FK	References batches(batch_id)
transporter_name	VARCHAR(100)	—	Name of logistics provider / carrier
vehicle_number	VARCHAR(30)	—	Transport vehicle license registration
pickup_location	VARCHAR(150)	—	Origin farm gate / storage facility
drop_location	VARCHAR(150)	—	Destination market terminal / APMC
dispatch_date	DATE	—	Date fleet departed origin
transit_status	ENUM(...)	—	Scheduled, In Transit, Reached, Delayed

Table 2.5: Retail Orders Table Schema

Field	Type	Key	Description
order_id	INT	PK	Unique order ID (Auto Increment)
batch_id	VARCHAR(50)	FK	References batches(batch_id)
retailer_name	VARCHAR(120)	—	Purchasing supermarket / retail entity
retailer_city	VARCHAR(100)	—	City of destination retail outlet
quantity_ordered_kg	DECIMAL(10,2)	—	Purchased volume in kilograms
selling_price_per_kg	DECIMAL(8,2)	—	Selling price per kilogram
total_amount	DECIMAL(12,2)	—	Computed order revenue (INR)
order_status	ENUM(...)	—	Pending, Confirmed, Dispatched, Delivered

2.5 Database Implementation: SQL DDL Code

The Data Definition Language (DDL) creates the agri_supply_chain database and defines relational tables with primary keys, foreign keys with ON DELETE CASCADE actions, and ENUM constraint domains:

SQL DDL: Database, Farmers, Crops, and Batches Creation

```
-- 1. Database Creation & Scope
CREATE DATABASE IF NOT EXISTS agri_supply_chain;
USE agri_supply_chain;

-- 2. Farmers Table Definition
CREATE TABLE farmers (
    farmer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    location VARCHAR(150) NOT NULL,
    farm_size_acres DECIMAL(6,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Crops Catalog Table
CREATE TABLE crops (
    crop_id INT AUTO_INCREMENT PRIMARY KEY,
    crop_name VARCHAR(100) NOT NULL,
    category ENUM('Grains', 'Vegetables', 'Fruits', 'Pulses', 'Spices', 'Cash Crops') DEFAULT 'Grains',
    base_price_per_kg DECIMAL(8,2) NOT NULL
);

-- 4. Batches Table (Core Unit with Cascading Constraints)
CREATE TABLE batches (
    batch_id VARCHAR(50) PRIMARY KEY,
    farmer_id INT NOT NULL,
    crop_id INT NOT NULL,
    quantity_kg DECIMAL(10,2) NOT NULL,
    harvest_date DATE NOT NULL,
    quality_grade ENUM('Grade A (Export)', 'Grade B (Premium)', 'Grade C (Standard)') DEFAULT 'Grade A
(Export)',
    current_status ENUM('Harvested', 'Cold Storage', 'In Transit', 'Distributed', 'Retail Shelved',
'Sold') DEFAULT 'Harvested',
    storage_temp_celsius DECIMAL(4,1) DEFAULT 18.0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (farmer_id) REFERENCES farmers(farmer_id) ON DELETE CASCADE,
    FOREIGN KEY (crop_id) REFERENCES crops(crop_id) ON DELETE CASCADE
);
```

2.5 DDL Code (Continued) & 2.6 DML Data Population

SQL DDL: Logistics and Retail Orders Tables

```
-- 5. Logistics Fleet Tracking Table
CREATE TABLE logistics (
    logistics_id INT AUTO_INCREMENT PRIMARY KEY,
    batch_id VARCHAR(50) NOT NULL,
    transporter_name VARCHAR(100) NOT NULL,
    vehicle_number VARCHAR(30) NOT NULL,
    pickup_location VARCHAR(150) NOT NULL,
    drop_location VARCHAR(150) NOT NULL,
    dispatch_date DATE NOT NULL,
    arrival_date DATE,
    transit_status ENUM('Scheduled', 'In Transit', 'Reached Destination', 'Delayed') DEFAULT 'Scheduled',
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id) ON DELETE CASCADE
);

-- 6. Retail Supermarket Orders Table
CREATE TABLE retail_orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    batch_id VARCHAR(50) NOT NULL,
    retailer_name VARCHAR(120) NOT NULL,
    retailer_city VARCHAR(100) NOT NULL,
    quantity_ordered_kg DECIMAL(10,2) NOT NULL,
    selling_price_per_kg DECIMAL(8,2) NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    order_date DATE NOT NULL,
    order_status ENUM('Pending', 'Confirmed', 'Dispatched', 'Delivered') DEFAULT 'Confirmed',
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id) ON DELETE CASCADE
);
```

2.6 Sample Data Population: SQL DML Code

SQL DML: Data Population Script

```
-- Seed Data for Farmers & Crops
INSERT INTO farmers (name, phone, location, farm_size_acres) VALUES
('Ramesh Kumar', '+91 98765 43210', 'Punjab, District Ludhiana', 15.5),
('Sita Devi', '+91 98112 23344', 'Nashik, Maharashtra', 8.2),
('Balwinder Singh', '+91 94567 89012', 'Karnal, Haryana', 22.0);

INSERT INTO crops (crop_name, category, base_price_per_kg) VALUES
('Basmati Rice 1121', 'Grains', 85.00),
('Organic Wheat (Sharbati)', 'Grains', 42.00),
('Alphonso Mango', 'Fruits', 250.00);

-- Seed Data for Batches & Logistics
INSERT INTO batches (batch_id, farmer_id, crop_id, quantity_kg, harvest_date, quality_grade,
current_status) VALUES
('BATCH-2026-RICE-01', 1, 1, 5000.00, '2026-10-15', 'Grade A (Export)', 'In Transit'),
('BATCH-2026-WHEAT-02', 3, 2, 8500.00, '2026-10-18', 'Grade A (Export)', 'Cold Storage');

INSERT INTO logistics (batch_id, transporter_name, vehicle_number, pickup_location, drop_location,
dispatch_date, transit_status) VALUES
('BATCH-2026-RICE-01', 'Kisan Express Cargo', 'PB-10-CX-4521', 'Ludhiana Mandi', 'Delhi Central Hub',
'2026-10-27', 'In Transit');
```

2.7 Relational Join Queries: SQL DQL Code

The following queries demonstrate the execution of multi-table joins, status aggregations, and traceability retrieval used by the application backend:

SQL DQL: Traceability, Aggregations, and Logistics Queries

```
-- Query 1: Farm-to-Table Multi-Table Traceability Join
-- Resolves complete audit trail from farmer origin to in-transit logistics carrier
SELECT
    b.batch_id,
    c.crop_name,
    f.name AS farmer_name,
    f.location AS farm_location,
    b.quantity_kg,
    b.harvest_date,
    b.quality_grade,
    b.storage_temp_celsius,
    b.current_status,
    l.transporter_name,
    l.vehicle_number,
    l.pickup_location,
    l.drop_location,
    l.transit_status
FROM batches b
JOIN farmers f ON b.farmer_id = f.farmer_id
JOIN crops c ON b.crop_id = c.crop_id
LEFT JOIN logistics l ON b.batch_id = l.batch_id
WHERE b.batch_id = 'BATCH-2026-RICE-01';

-- Query 2: Aggregate Valuation & Crop Volume Analytics
SELECT
    c.category,
    COUNT(b.batch_id) AS total_batches,
    SUM(b.quantity_kg) AS total_harvested_volume_kg,
    AVG(b.storage_temp_celsius) AS avg_cold_storage_temp
FROM batches b
JOIN crops c ON b.crop_id = c.crop_id
GROUP BY c.category
HAVING total_harvested_volume_kg > 1000;

-- Query 3: Real-Time Active Transit Fleets Monitoring
SELECT
    l.logistics_id,
    l.batch_id,
    l.transporter_name,
    l.vehicle_number,
    CONCAT(l.pickup_location, ' -> ', l.drop_location) AS route,
    l.dispatch_date,
    l.transit_status
FROM logistics l
WHERE l.transit_status = 'In Transit'
ORDER BY l.dispatch_date DESC;
```

Chapter 3
Frontend Implementation and Features

3.1 System Architecture Overview

The Agricultural Supply Chain Management System provides users with a centralized, responsive interface for managing and verifying agricultural products. The application replicates enterprise agricultural logistics systems while showcasing practical relational database integration.

The system employs a 3-tier architecture: the frontend client communicates with a native Node.js REST API layer that queries a persistent MySQL relational database. Concurrently, a standalone Java Desktop and JApplet client connects directly through JDBC to perform desktop-grade CRUD operations.

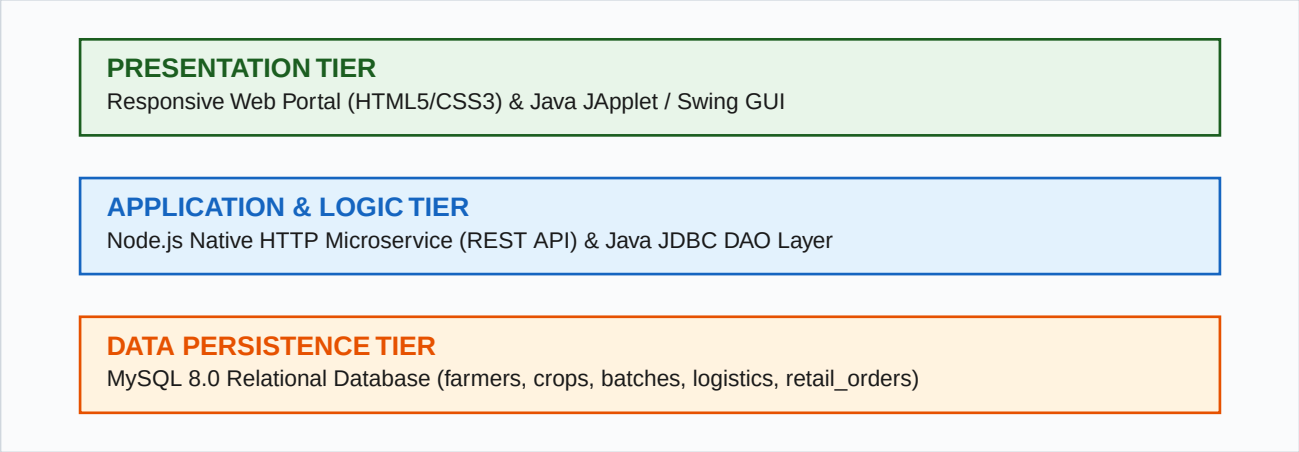

Figure 3.1: 3-Tier System Architecture Diagram

3.2 Live Supply Chain Operations Dashboard

The live dashboard serves as the central control room for supply chain supervisors. It aggregates live operational statistics into four metric cards and renders an interactive data table displaying all active batches in the supply stream.

Figure 3.2: Live Operations Dashboard – Inventory and KPI View

3.3 Farmer Harvest Registration Feature

The Farmer Harvest Registration module allows agricultural producers and mandi inspectors to log fresh harvest yields directly into the database. Every harvest is assigned an immutable Batch ID, linked to the registered farmer, categorized by crop type, and inspected for quality grade (Grade A Export, Grade B Domestic Premium, Grade C Standard Processing).

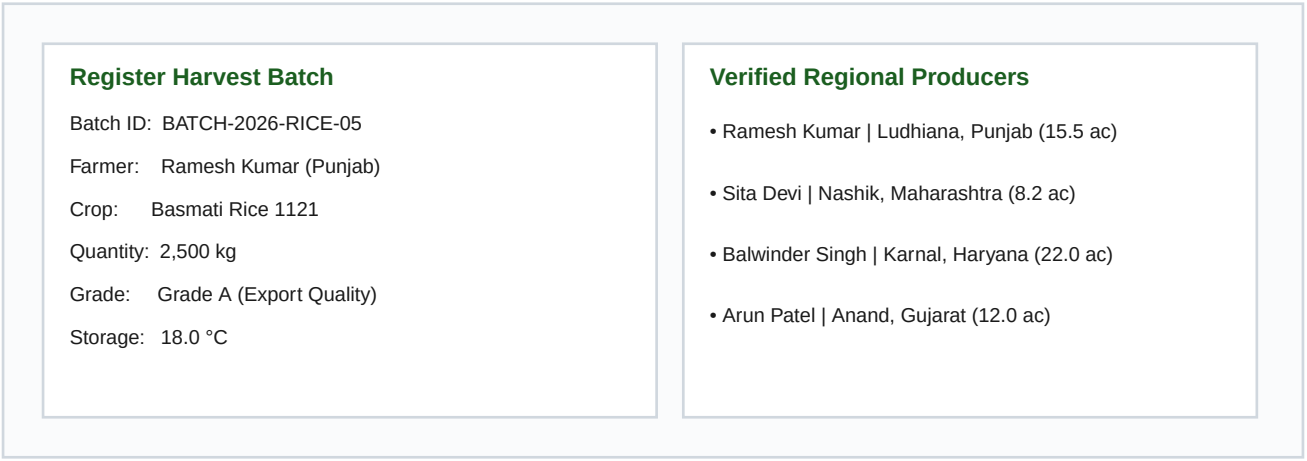

Figure 3.3: Farmer Harvest Registration and Producer Directory

3.4 Cold Chain Logistics and Fleet Transit

Temperature abuse during transit causes rapid decay of perishable crops. The Cold Logistics module enables operators to dispatch refrigerated trucks, record the driver agency, vehicle registration plate, origin farm gate, destination wholesale APMC, and monitored temperature.

Dispatching a carrier automatically updates the relational batch state from 'Harvested' or 'Cold Storage' to 'In Transit', preventing double-dispatch anomalies.

3.5 Retail Hub and Supermarket Orders

The Retail Hub facilitates B2B commercial transactions between supply hubs and commercial supermarket chains (e.g. Nature Fresh, Daily Greens). Supermarket procurement managers select authenticated available batches, specify purchase volume (kg), and agree upon wholesale rates.

The system calculates total transaction amounts, logs the purchase record into the retail_orders table, and marks the batch status as 'Sold' or fulfilled upon confirmation.

3.6 Farm-to-Table Traceability & 3.7 Java JDBC Code

The consumer portal executes multi-table joins to render an authenticated audit trail. Below is the Java Database Connectivity (JDBC) implementation from DBConnection.java and AgriSupplyChainApp.java:

Java JDBC: Connection Driver & Parameterized PreparedStatements

```
// 1. Database Connection Management (DBConnection.java)
public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/agri_supply_chain?useSSL=false";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}

// 2. Parameterized SQL Execution in JApplet GUI (AgriSupplyChainApp.java)
String sql = "INSERT INTO batches (batch_id, farmer_id, crop_id, quantity_kg, "
           + "harvest_date, quality_grade, current_status, storage_temp_celsius) "
           + "VALUES (?, ?, ?, ?, CURDATE(), ?, ?, ?)";

try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, batchId);
    stmt.setInt(2, Integer.parseInt(farmerIdStr));
    stmt.setInt(3, Integer.parseInt(cropIdStr));
    stmt.setDouble(4, Double.parseDouble(qtyStr));
    stmt.setString(5, (String) cmbGrade.getSelectedItem());
    stmt.setString(6, (String) cmbStatus.getSelectedItem());
    stmt.setDouble(7, Double.parseDouble(tempStr));
    stmt.executeUpdate(); // Zero SQL injection risk
}
```

Chapter 4
Conclusion & Future Scope

4.1 Summary of Project Work

The Agricultural Supply Chain Management System project was developed to provide a robust, transparent, and user-friendly platform for managing agricultural logistics. The project successfully combines frontend web and Java development with relational database management using MySQL.

The database was designed using MySQL and consists of five tables: Farmers, Crops, Batches, Logistics, and Retail Orders. Primary keys, foreign keys, and ENUM domain constraints were implemented to maintain data integrity and establish relationships between the tables.

Through this project, practical knowledge of database design, SQL DDL/DML/DQL queries, relational schema development, and frontend-to-backend connectivity was applied. The project demonstrates how relational database concepts can be integrated into an agricultural application to organize and manage supply chain information effectively.

4.2 Achievement of Project Objectives

The main objectives of the Agricultural Supply Chain Management System project were achieved by developing an interactive interface integrated with a relational database:

1. User-Friendly Interface: Developed a modern web interface with tab navigation for convenient access to different operational sections of the application.
2. Harvest & Crop Registration: Implemented structured forms for registering new harvest batches with quality grades and monitored temperatures.
3. Cold Chain Management: Integrated logistics dispatch functionality to monitor refrigerated carriers and transit routes.
4. Retail Hub: Provided procurement ordering workflows with dynamic revenue calculation.
5. Consumer Traceability: Built an instant multi-table relational query feature allowing consumers to verify product history by Batch ID.
6. Database Design & Integrity: Designed a 5-table MySQL schema utilizing primary keys, foreign keys, and cascade constraints to prevent data anomalies.
7. Java JDBC Connectivity: Implemented standalone Java classes to execute parameterized SQL statements via JDBC drivers.

4.3 Future Enhancements

While the current version of the Agricultural Supply Chain Management System successfully implements core database and logistics tracking features, several advanced enhancements are planned for future versions:

• IoT Hardware Telematics: Direct integration of GPS trackers and digital thermistors inside refrigerated cargo vehicles to stream live temperature data directly to the database via MQTT/HTTP protocols.

• Blockchain Audit Trail: Anchoring critical batch lifecycle transactions to an immutable distributed ledger (e.g. Ethereum or Hyperledger Fabric) to prevent retrospective record tampering.

• Dynamic QR Code Generation: Automated generation of printable GS1-standardized QR stickers for immediate scanning by consumer smartphones at retail checkout counters.

• Automated Payment Escrow: Integration of UPI and banking payment gateways to automatically disburse funds to farmers upon digital signature of receipt by retail buyers.

References & Bibliography

1. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th ed.). Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts (7th ed.). McGraw-Hill.
3. Oracle Corporation. MySQL 8.0 Reference Manual: Relational Database Architecture and SQL Syntax.
4. Horstmann, C. S. Core Java Volume I – Fundamentals (12th ed.). Prentice Hall.